

รายงาน Mini Project

การพัฒนาเว็บไซต์แบบ Full-Stack ด้วย REST API

Workout Routine Catalog

ระบบจัดการตารางออกกำลังกาย

รายวิชา

SELECTED TOPICS IN ELECTRONICS AND COMPUTER ENGINEERING I

จัดทำโดย

ชัชณพงศ์ เจริญจิตรรัตน์ 660910488

ศิววัฒน์ มกรกิจวิบูลย์ 660910505

ชินภัทร ศุภพฤษพงษ์ 660910538

บทนำ

การพัฒนาเว็บไซต์ในปัจจุบันไม่ได้จำกัดอยู่เพียงการสร้างหน้าเว็บไซต์เท่านั้น แต่ยังรวมถึงการพัฒนาระบบฝั่งเซิร์ฟเวอร์สำหรับการจัดการข้อมูลและให้บริการแก่ฝั่งไคลเอนต์ การพัฒนาเว็บไซต์แบบ Full-Stack จึงเป็นการทำงานร่วมกันระหว่าง Frontend และ Backend เพื่อให้ผู้ใช้งานสามารถโต้ตอบกับข้อมูลภายในระบบได้อย่างมีประสิทธิภาพ

Mini Project นี้จัดทำขึ้นเพื่อฝึกปฏิบัติการพัฒนาเว็บไซต์แบบ Full-Stack โดยเลือกหัวข้อ Workout Routine Catalog ซึ่งเป็นระบบสำหรับการจัดการข้อมูลตารางและท่าออกกำลังกาย ระบบมี REST API สำหรับจัดการข้อมูลในรูปแบบ CRUD และมีส่วนติดต่อผู้ใช้งานที่พัฒนาด้วย HTML, CSS และ Vanilla JavaScript โดยใช้ Fetch API สำหรับรับส่งข้อมูลระหว่าง Client และ Server

ระบบใช้ Node.js และ Express เป็น Backend และใช้ฐานข้อมูลแบบ In-Memory Array สำหรับจัดเก็บข้อมูล พร้อมข้อมูลตั้งต้นจำนวน 8 รายการ เมื่อเริ่มต้นเซิร์ฟเวอร์ ระบบจะโหลดข้อมูลตัวอย่างสำหรับท่าออกกำลังกาย เช่น Push-up, Bench Press, Squat, Pull-up, Plank, Deadlift, Burpee และ Lunges

2. วัตถุประสงค์

1. เพื่อศึกษาการพัฒนาเว็บไซต์แบบ Full-Stack
2. เพื่อศึกษาการสร้าง REST API ด้วย Node.js และ Express.js
3. เพื่อฝึกการเชื่อมต่อระหว่าง Frontend และ Backend
4. เพื่อฝึกการจัดการข้อมูลด้วย REST API ในรูปแบบ CRUD
5. เพื่อฝึกการใช้ HTTP Methods เช่น GET, POST, PATCH และ DELETE
6. เพื่อพัฒนาเว็บแอปพลิเคชันสำหรับจัดการข้อมูลท่าออกกำลังกายที่สามารถใช้งานฟังก์ชันต่าง ๆ ได้จริง
7. เพื่อทดสอบและตรวจสอบการทำงานของ API และส่วนต่าง ๆ ของเว็บไซต์

3. รายละเอียดของระบบ

Workout Routine Catalog เป็นเว็บแอปพลิเคชันสำหรับจัดการตารางและข้อมูลท่าออกกำลังกาย โดยระบบประกอบด้วยส่วน Client และ Server ผู้ใช้งานสามารถดูรายการท่าออกกำลังกาย เพิ่มรายการ แก้ไขข้อมูล ลบรายการ เปลี่ยนสถานะ และกรองหรือค้นหาข้อมูลตามเงื่อนไขที่กำหนด

ส่วนประกอบ	รายละเอียด
Backend	Node.js + Express และ REST API
Frontend	HTML5 + CSS3 + Vanilla JavaScript
ฐานข้อมูล	In-Memory Array
ข้อมูลตั้งต้น	8 รายการ
External API	ไม่มี ใช้ API ของระบบเอง
Port	3000 (สามารถเปลี่ยนผ่าน PORT environment variable)

3.1 ฟีเจอร์หลัก

- REST API รองรับการทำงาน CRUD สำหรับข้อมูลท่าออกกำลังกาย
- รองรับการกรองข้อมูลด้วย muscle, level, equipment, status และ q
- ตรวจสอบข้อมูลทั้งฝั่ง Client และ Server
- คืน HTTP Status Code ที่เหมาะสม เช่น 200, 201, 204, 400 และ 404
- มีข้อมูลตั้งต้น 8 ท่าออกกำลังกาย
- มี Endpoint /api/meta สำหรับค่าที่ใช้สร้าง Dropdown

Frontend อัปเดตรายการผ่าน Fetch API โดยไม่ต้องรีเฟรชหน้า

- มี Presets สำหรับเลือกทำสำเร็จรูปและเติมข้อมูลลงในฟอร์ม
- มีฟังก์ชันเพิ่ม แก้ไข เปลี่ยนสถานะ และลบรายการ

4. เทคโนโลยีที่ใช้

4.1 Backend

- Node.js — Runtime สำหรับทำงานฝั่ง Server
- Express.js — Web Framework และระบบ Routing สำหรับ REST API
- `express.json()` — ใช้รับข้อมูล JSON จาก Request
- In-Memory Array — ใช้เป็นฐานข้อมูลจำลองสำหรับเก็บข้อมูลในหน่วยความจำ
- CORS Header แบบกำหนดเอง — รองรับการทดสอบ API ข้ามเครื่อง

4.2 Frontend

- HTML5 Semantic + ARIA — โครงสร้างหน้าเว็บไซต์และการเข้าถึง
- CSS3 — CSS Variables, Grid/Flexbox, Animation และ Responsive Design
- Vanilla JavaScript — ใช้ `fetch()`, `async/await`, `URLSearchParams` และ Event Delegation

4.3 เครื่องมือและแพ็คเกจ

- Visual Studio Code — เครื่องมือสำหรับเขียนและแก้ไขโปรแกรม
- Node.js และ npm — ใช้ติดตั้งและจัดการแพ็คเกจ
- Express 4 — Dependency หลักของระบบ
- Git/GitHub — ใช้สำหรับจัดเก็บและจัดการซอร์สโค้ดของโครงการ

5. โครงสร้างการทำงานของระบบ

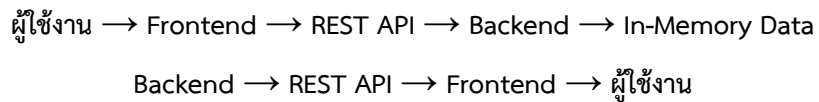
ระบบแบ่งการทำงานออกเป็น 2 ส่วนหลัก ได้แก่ Client และ Server โดย Client เป็นส่วนที่ผู้ใช้งาน มองเห็นและโต้ตอบโดยตรง ส่วน Server ทำหน้าที่ให้บริการ REST API ประมวลผล Request ตรวจสอบข้อมูล และส่ง Response กลับไปยัง Client

5.1 Client

Client ทำหน้าที่แสดงรายการทำออกกำลังกาย รับข้อมูลจากแบบฟอร์ม แสดงตัวกรองและช่องค้นหา รวมถึงส่ง Request ไปยัง REST API ผ่าน Fetch API เมื่อผู้ใช้งานเพิ่ม แก้ไข เปลี่ยนสถานะ หรือลบข้อมูล

5.2 Server

Server พัฒนาด้วย Node.js และ Express โดยทำหน้าที่จัดการ Routing ของ API ตรวจสอบข้อมูล อ่านและแก้ไขข้อมูลใน In-Memory Array และส่ง HTTP Status Code พร้อม Response กลับไปยัง Client



6. โครงสร้างโปรเจกต์

โครงสร้างหลักของโปรเจกต์มีลักษณะดังนี้

Fitness_Routine/

├── package.json

├── package-lock.json

├── README.md

├── server/

│ └── server.js

└── public/

├── index.html

├── style.css

└── app.js

ไฟล์ package.json ใช้กำหนดชื่อโปรเจกต์ สคริปต์สำหรับรันระบบ และ dependency โดยโปรเจกต์ใช้ Express เป็น dependency หลัก และกำหนดคำสั่ง npm start สำหรับรัน server รวมถึง npm run dev สำหรับโหมดที่มีการ restart server อัตโนมัติเมื่อแก้ไขไฟล์

6.1 หน้าที่ของไฟล์สำคัญ

ไฟล์/โฟลเดอร์	หน้าที่
server/server.js	Express server, REST API, Memory DB และ Seed Data
public/index.html	โครงสร้างหน้า UI เช่น ฟорм ตัวกรอง และการดรากรายการ
public/style.css	รูปแบบหน้าตา ธีม Responsive และ Design Tokens
public/app.js	Presets, Fetch CRUD, Render และ Toast
package.json	กำหนด dependency และคำสั่ง npm scripts
README.md	เอกสารอธิบายการติดตั้ง การใช้งาน API และการทดสอบ

7. การออกแบบ REST API

ระบบ Workout Routine Catalog ออกแบบ REST API เพื่อให้ Client สามารถติดต่อกับ Server ผ่าน HTTP Request โดย Resource หลักของระบบคือ workouts และมี Endpoint เสริมสำหรับ meta

HTTP Method	หน้าที่
GET	เรียกดูข้อมูล
POST	เพิ่มข้อมูล
PATCH	แก้ไขข้อมูลบางส่วน
DELETE	ลบข้อมูล

7.1 Resource หลัก

- Workouts — จัดการข้อมูลท่าออกกำลังกาย
- Meta — ให้ค่าตัวเลือกที่ใช้สร้าง Dropdown

Base URL ของ API คือ <http://localhost:3000/api>

8. รายละเอียด Endpoint ของระบบ

Method	Endpoint	คำอธิบาย	Status
GET	/api/workouts	ดึงรายการท่าออกกำลังกายทั้งหมด รองรับ filter	200
GET	/api/workouts/:id	ดึงข้อมูลตาม ID	200 / 404
POST	/api/workouts	เพิ่มท่าออกกำลังกายใหม่	201 / 400
PATCH	/api/workouts/:id	แก้ไขข้อมูลบางส่วน	200 / 404 / 400
DELETE	/api/workouts/:id	ลบรายการ	204 / 404
GET	/api/meta	ดึงค่า enum สำหรับ Dropdown	200

8.1 ตัวอย่างการทำงาน

GET /api/workouts ใช้สำหรับเรียกดูรายการทั้งหมด

POST /api/workouts ใช้สำหรับเพิ่มข้อมูล เช่น ชื่อท่า กลุ่มกล้ามเนื้อ ระดับความยาก อุปกรณ์ จำนวนเซต และจำนวนครั้ง

PATCH /api/workouts/:id ใช้แก้ไขข้อมูลบางส่วน เช่น เปลี่ยน status หรือเปลี่ยน sets, reps และ level

DELETE /api/workouts/:id ใช้ลบข้อมูลตาม ID

9. การกรองข้อมูลและ Query Parameters

GET /api/workouts รองรับกรกรองข้อมูลหลายเงื่อนไข และสามารถใช้หลายเงื่อนไขรวมกันได้

พารามิเตอร์	ค่าที่ได้รับได้	ตัวอย่าง
muscle	chest, back, legs, core, shoulders, arms, full-body	/api/workouts?muscle=chest
level	beginner, intermediate, advanced	/api/workouts?level=beginner
equipment	ค่าของอุปกรณ์ เช่น bodyweight, barbell	/api/workouts?equipment=barbell
status	active, paused, archived	/api/workouts?status=active
q	ค้นหาจากชื่อ/รายละเอียด/อุปกรณ์	/api/workouts?q=squat

ตัวอย่างการใช้หลายเงื่อนไข: /api/workouts?muscle=legs&level=beginner

10. HTTP Status Code

ระบบใช้ HTTP Status Code เพื่อแสดงผลลัพธ์ของ Request ที่ส่งเข้ามา

Status Code	ความหมาย	การใช้งานในระบบ
200 OK	Request สำเร็จ	เรียกดูหรือแก้ไขข้อมูลสำเร็จ
201 Created	สร้างข้อมูลสำเร็จ	เพิ่มรายการทำออกกำลังกายสำเร็จ
204 No Content	ไม่มีข้อมูลใน Response	ลบข้อมูลสำเร็จ
400 Bad Request	ข้อมูลไม่ถูกต้อง/ไม่ครบ	ข้อมูลที่ส่งไม่ผ่าน Validation
404 Not Found	ไม่พบข้อมูล	ไม่พบรายการตาม ID

11. การทำงานของฝั่ง Client

ฝั่ง Client ใช้ HTML5, CSS3 และ Vanilla JavaScript ในการสร้างหน้าเว็บและจัดการการโต้ตอบกับผู้ใช้งาน โดยใช้ Fetch API ติดต่อกับ REST API เมื่อมีการเพิ่ม แก้ไข เปลี่ยนสถานะ หรือลบข้อมูล

1. ผู้ใช้งานเลือก Preset หรือกรอกข้อมูลในฟอร์ม
2. Client ตรวจสอบข้อมูลเบื้องต้นก่อนส่ง Request
3. JavaScript ส่ง Request ผ่าน Fetch API ไปยัง Endpoint ที่เกี่ยวข้อง
4. Server ตรวจสอบข้อมูลและประมวลผลกับ In-Memory Array
5. Server ส่ง Response พร้อม HTTP Status Code กลับมา
6. Client อ่าน Response และอัปเดตรายการบนหน้าเว็บทันที

การอัปเดตข้อมูลทำโดยไม่จำเป็นต้องโหลดหน้าเว็บไซต์ใหม่ทุกครั้ง

12. ฟังก์ชันการทำงานของเว็บไซต์

12.1 การแสดงรายการทำออกกำลักาย

ระบบเรียกข้อมูลจาก REST API และแสดงผลเป็นการ์ด พร้อมข้อมูลกลุ่มกล้ามเนื้อ ระดับความยาก อุปกรณ์ และรายละเอียด

12.2 Presets Select

มีคลังทำสำเร็จรูป 10 ทำในส่วนของ Client เมื่อเลือกแล้วระบบสามารถเติมข้อมูลลงในฟอร์มอัตโนมัติ

12.3 การเพิ่มข้อมูล

ผู้ใช้งานกรอกข้อมูลในฟอร์มและกดบันทึก ระบบส่ง HTTP POST ไปยัง REST API

12.4 การแก้ไขข้อมูล

ผู้ใช้งานเลือกแก้ไข ระบบส่ง HTTP PATCH เพื่อแก้ไขข้อมูลบางส่วน

12.5 การเปลี่ยนสถานะ

ระบบรองรับการเปลี่ยนสถานะ active → paused → archived ผ่าน PATCH

12.6 การลบข้อมูล

ผู้ใช้งานกดลบ ระบบแสดง Modal ยืนยัน จากนั้นส่ง HTTP DELETE

12.7 การกรองและค้นหา

สามารถกรองตามกลุ่มกล้ามเนื้อ ระดับความยาก สถานะ อุปกรณ์ และค้นหาด้วยคำค้น

12.8 การแจ้งเตือน

ระบบใช้ Toast Notification เพื่อแจ้งผลการทำงานแก่ผู้ใช้งาน

13. การออกแบบข้อมูลและ Validation

ข้อมูลทำออกกำลังกายถูกจัดเก็บเป็น Object ภายใน Array โดยมีโครงสร้างหลักดังนี้

Field	รายละเอียด
id	เลข ID สร้างอัตโนมัติ
name	ชื่อทำออกกำลังกาย จำเป็นต้องมี
muscle	กลุ่มกล้ามเนื้อ
level	ระดับ beginner / intermediate / advanced
equipment	อุปกรณ์ที่ใช้
sets	จำนวนเซต 1-20
reps	จำนวนครั้ง 1-200
duration	ระยะเวลาดังเป็นวินาที โดย 0 หมายถึงนับ reps
description	คำอธิบายวิธีทำ
status	active / paused / archived
createdAt	เวลาที่สร้างข้อมูล

13.1 การตรวจสอบข้อมูล 2 ชั้น

- Client-side validation ตรวจสอบบังคับ ชนิดข้อมูล และช่วงตัวเลขก่อนส่ง
- Server-side validation ตรวจสอบด้วยฟังก์ชัน `validateWorkout()` เพื่อให้การตรวจสอบที่ Server เป็นตัวควบคุมหลัก

ฟิลด์บังคับ ได้แก่ name, muscle, level, equipment, sets, reps และ description ส่วน muscle, level และ status ต้องเป็นค่าที่ระบบกำหนด และมีการตรวจสอบช่วงของ sets, reps และ duration

14. การติดตั้งและการใช้งานระบบ

เครื่องมือที่ใช้พัฒนาควรมี Node.js เวอร์ชัน 18 ขึ้นไปและ npm

9. เปิด Terminal แล้วเข้าไปยังโฟลเดอร์ Fitness_Routine
10. ติดตั้ง dependency ด้วยคำสั่ง npm install
11. รันระบบด้วยคำสั่ง npm start หรือ npm run dev
12. เปิดเบราว์เซอร์ไปที่ <http://localhost:3000>

คำสั่งหลัก

```
npm install
```

```
npm start
```

```
npm run dev
```

เมื่อเริ่มต้นระบบจะมีข้อความแสดงว่า Server ทำงานที่ <http://localhost:3000> และมี Seed Data จำนวน 8 รายการ

เนื่องจากระบบใช้ In-Memory DB ข้อมูลที่ผู้ใช้งานเพิ่มหรือแก้ไขจะไม่ถูกเก็บถาวร และจะกลับไปเป็น Seed Data

เมื่อปิดและเปิดเซิร์ฟเวอร์ใหม่

15. การทดสอบระบบ

หลังจากพัฒนาระบบ ได้กำหนดกรณีทดสอบเพื่อยืนยันการทำงานของหน้าเว็บไซต์และ REST API

#	กรณีทดสอบ	ผลลัพธ์ที่คาดหวัง
1	เปิด http://localhost:3000	เห็นการ์ด Seed Data 8 รายการ
2	เลือก Preset Squat	ฟอร์มถูกเติมข้อมูลอัตโนมัติ
3	กดบันทึกโดยไม่กรอกข้อมูล	Toast แจ้งเตือนและช่องชื่อถูกไฮไลต์
4	กรอกข้อมูลครบแล้วบันทึก	เพิ่มรายการใหม่ทันทีโดยไม่มีเฟรช
5	เลือก filter Chest	เหลือเฉพาะรายการกลุ่มอก
6	ค้นหา squat	แสดงรายการที่ตรงกับคำค้น
7	เปลี่ยนสถานะ	สถานะเปลี่ยนตามลำดับที่ระบบกำหนด
8	แก้ไขข้อมูล	ข้อมูลในการ์ดเปลี่ยนตาม PATCH
9	ลบข้อมูล	ยืนยันแล้วรายการหายไปและ API ตอบ 204
10	GET /api/workouts/999	ตอบ 404 Not Found
11	POST ข้อมูลไม่ครบ	ตอบ 400 Bad Request
12	ปิด-เปิด Server	ข้อมูลกลับเป็น Seed Data 8 รายการ

16. ผลการทดสอบ CRUD

16.1 Create

สามารถเพิ่มข้อมูลผ่าน HTTP POST ได้ และเมื่อข้อมูลถูกต้องระบบตอบกลับด้วย HTTP Status Code 201 Created

16.2 Read

สามารถเรียกดูรายการทั้งหมดและเรียกดูข้อมูลตาม ID ได้ โดยเมื่อพบข้อมูลระบบตอบกลับด้วย HTTP Status Code 200 OK

16.3 Update

สามารถแก้ไขข้อมูลด้วย HTTP PATCH และตรวจสอบกรณีที่ไม่มีพบข้อมูลได้

16.4 Delete

สามารถลบข้อมูลด้วย HTTP DELETE และเมื่อดำเนินการสำเร็จระบบตอบกลับด้วย HTTP Status Code 204 No Content

16.5 ผลการทดสอบ API กรณีข้อมูลผิดพลาด

เมื่อส่งข้อมูลไม่ครบตามที่กำหนด ระบบจะตอบกลับ 400 Bad Request และเมื่อค้นหา ID ที่ไม่มีอยู่ ระบบจะตอบกลับ 404 Not Found

17. ปัญหาและข้อจำกัดของระบบ

- ใช้ In-Memory DB ทำให้ข้อมูลหายเมื่อปิดเซิร์ฟเวอร์
- ยังไม่มีระบบ Authentication / Authorization
- ยังไม่มี Pagination จึงเหมาะกับข้อมูลจำนวนไม่มาก

ข้อจำกัดดังกล่าวเป็นข้อจำกัดของเวอร์ชันปัจจุบันตามขอบเขตของ Mini Project

และสามารถนำไปใช้เป็นแนวทางสำหรับการพัฒนาระบบในอนาคต

18. แนวทางพัฒนาต่อ

- เชื่อมต่อฐานข้อมูลจริง เช่น MongoDB หรือ MySQL แทน In-Memory DB
- เพิ่มระบบ Login และแยกสิทธิ์ผู้ใช้งาน
- เพิ่ม Pagination และ Sorting
- เพิ่มการอัปโหลดรูปภาพประจำตัวออกกำลังกาย
- เพิ่ม Unit Test และ Integration Test ด้วย Jest และ Supertest

19. สรุปผลการพัฒนา

จากการจัดทำ Mini Project เรื่อง Workout Routine Catalog สามารถพัฒนาเว็บแอปพลิเคชันแบบ Full-Stack ที่ประกอบด้วย Frontend และ Backend ได้ตามขอบเขตของโครงการ โดยมี REST API สำหรับจัดการข้อมูลท่าออกกำลังกาย และรองรับการทำงานในรูปแบบ CRUD

ระบบสามารถแสดง เพิ่ม แก้ไข ลบ และเปลี่ยนสถานะข้อมูล รวมถึงรองรับการกรองและค้นหาข้อมูลผ่าน Query Parameters โดยมีการตรวจสอบข้อมูลทั้งฝั่ง Client และ Server และใช้ HTTP Status Code เพื่อสื่อสารผลลัพธ์ของการทำงาน

จากการกำหนดกรณีทดสอบ ระบบสามารถรองรับการทำงานหลักตามที่ยกแบบไว้ เช่น การเพิ่มข้อมูล การเรียกดูข้อมูล การแก้ไข การลบ การกรองข้อมูล และการจัดการกรณีข้อมูลไม่ถูกต้องหรือไม่พบรายการ

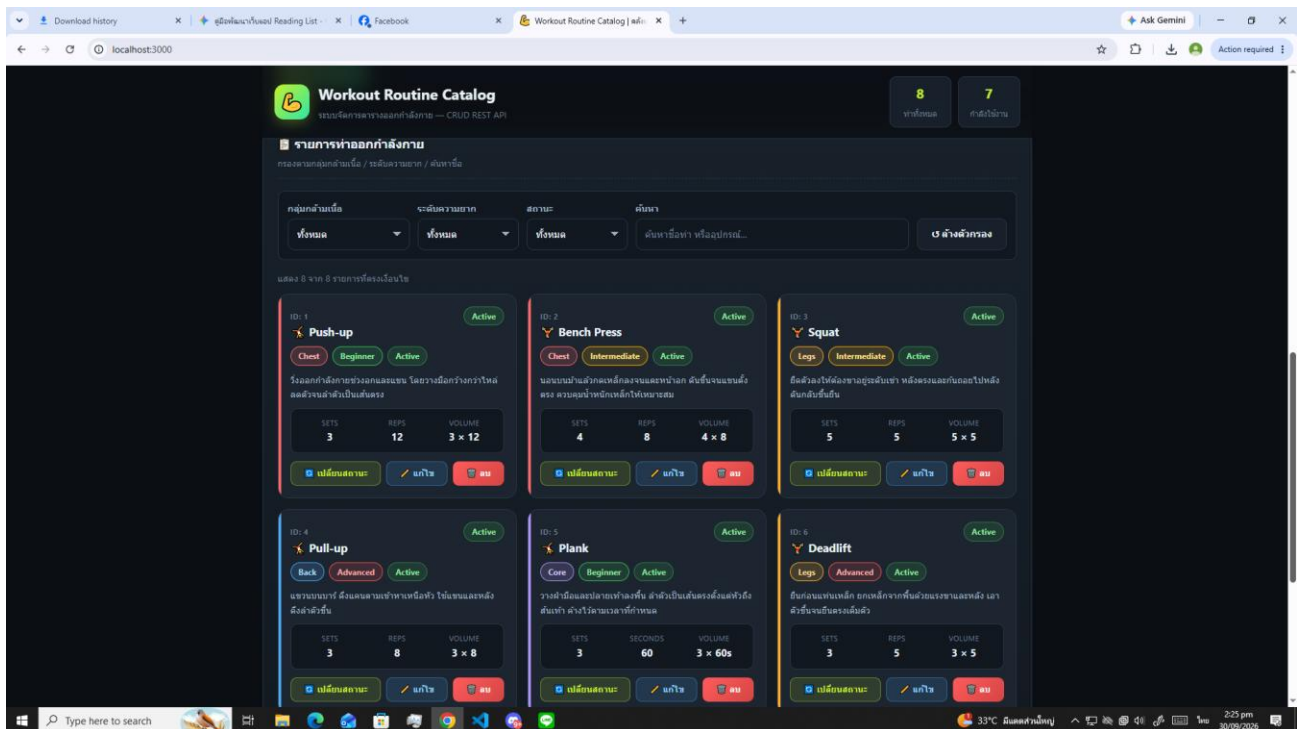
การจัดทำ Mini Project นี้ช่วยให้เกิดความเข้าใจเกี่ยวกับการพัฒนาเว็บไซต์แบบ Full-Stack การทำงานของ REST API การรับส่งข้อมูลระหว่าง Client และ Server การใช้ HTTP Methods และการออกแบบ Validation สำหรับระบบเว็บแอปพลิเคชัน

ภาคผนวก ก: ภาพการทำงานของเว็บไซต์

ส่วนนี้จัดเตรียมตำแหน่งสำหรับแทรกภาพหน้าจอจากชิ้นงานจริง

โดยควรใช้ภาพจากโปรเจกต์ของกลุ่มเพื่อแสดงผลการทำงานของระบบ

ภาพที่ ก-1 หน้าแสดงรายการ Workout Routine



ภาพที่ ก-2 หน้า/ส่วนฟอร์มเพิ่มข้อมูล

Download history

ดูประวัติการดาวน์โหลด Reading List

Facebook

Workout Routine Catalog | แอป

Ask Gemini

localhost:3000

Workout Routine Catalog

ระบบจัดการตารางออกกำลังกายแบบ CRUD REST API

8

ทำเสร็จแล้ว

7

กำลังฝึก

+ เพิ่มท่าออกกำลังกายใหม่

กรุณาใส่ข้อมูลให้ครบทุกช่องเพื่อให้ได้ผลลัพธ์ที่แม่นยำ * แล้วกดบันทึก

ชื่อท่า (name) *

เพิ่ม Push-up

กลุ่มกล้ามเนื้อ (muscle) *

-- เลือกกลุ่มกล้ามเนื้อ --

ระดับความยาก (level) *

-- เลือกระดับ --

อุปกรณ์ (equipment) *

เพิ่ม barbell / dumbbell

สถานะ (status) *

Active (กำลังใช้งาน)

จำนวนเซต (sets) *

3

จำนวนครั้ง (reps) *

12

ระยะเวลาพัก (interval) *

0 = ไม่พักเวลา

รายละเอียดท่า (description) *

อธิบายขั้นตอนการท่า...

บันทึกท่า

Type here to search

33°C

เมืองสาบสูญ

2:25 pm

30/05/2025

ภาพที่ ก-3 การแก้ไขข้อมูล

Workout Routine Catalog

8 ท่าออกกำลังกาย 7 ท่าออกกำลังกาย

Bench Press (Chest) แก้ไขข้อมูล ลบ

• Push-up • **Bench Press** • Squat • Pull-up • Plank • Burpee • Lunges • Deadlift • Shoulder Press

• Biceps Curl

+ เพิ่มท่าออกกำลังกายใหม่

กรุณากรอกข้อมูลให้ครบถ้วนก่อนกดปุ่ม "เพิ่มท่าออกกำลังกาย"

ชื่อท่า (name) *

Bench Press

กลุ่มกล้ามเนื้อ (muscle) *

Chest (an)

ระดับความยาก (level) *

Intermediate (ปานกลาง)

อุปกรณ์ (equipment) *

barbell

สถานะ (status)

Active (กำลังใช้งาน)

จำนวนเซต (sets) *

4

จำนวนครั้ง (reps) *

8

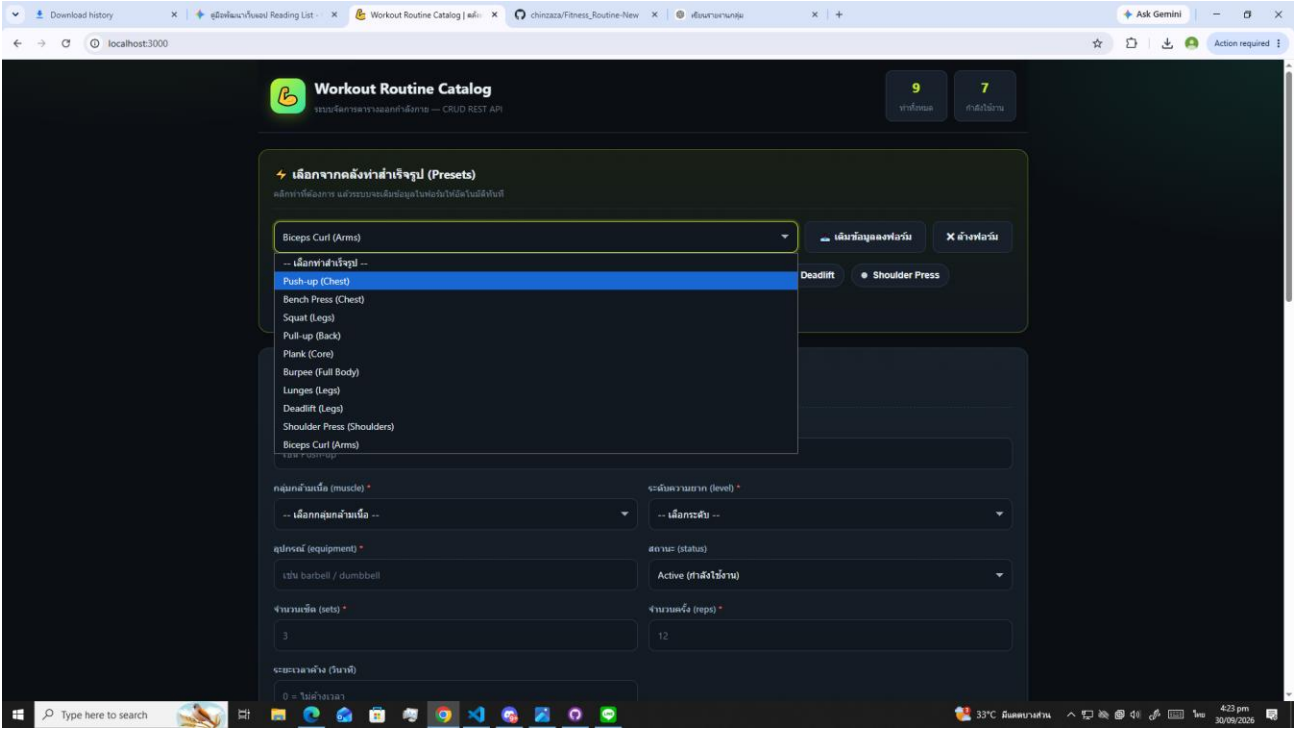
ระยะเวลาพัก (minutes)

0

รายละเอียดท่าออกกำลังกาย (description) *

นอนหงายบนเก้าอี้ จับบาร์เบลล์ไว้โดยยืนเหนือหน้าอก กดลงจนแตะอก สับซี่โครงและตรง

ภาพที่ ก-4 การกรองและค้นหาข้อมูล



ภาพที่ ก-5 การเปลี่ยนสถานะ

The screenshot displays the 'Workout Routine Catalog' application running on a web browser at localhost:3000. The interface features a dark theme and a grid of workout routine cards. Each card includes a title, difficulty level, and exercise details. On the right side, a vertical list of notifications shows the status changes for several routines.

Workout Routine Catalog
ระบบจัดการตารางออกกำลังกาย — CRUD REST API

Top Bar: 9 3 (จำนวน)

Workout Routines Grid:

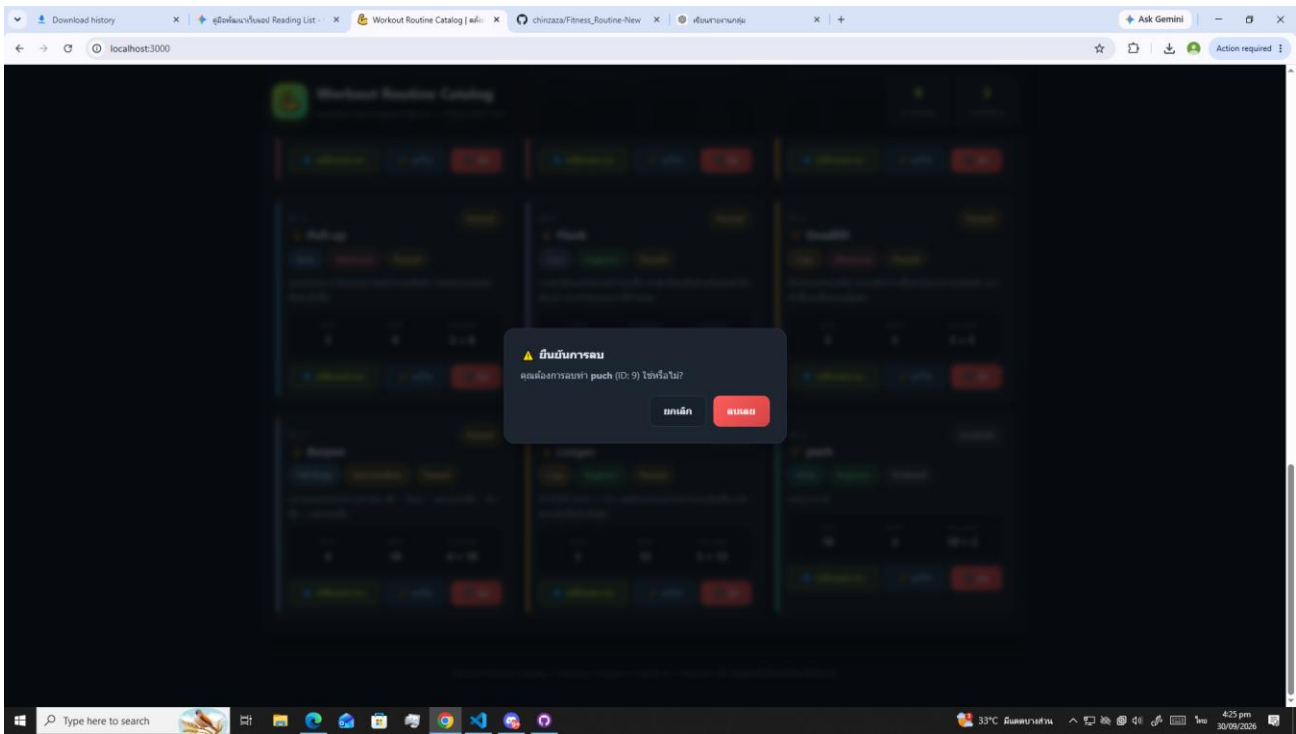
- ID: 4 Pull-up** (Paused)
Back Advanced Paused
แขยงแขนขึ้น ดึงแขนเข้าหาหน้าอก หัวไหล่ แขน และหลัง
ดึงลำตัวขึ้น
SETS: 3 REPS: 8 VOLUME: 3 x 8
- ID: 5 Plank** (Paused)
Core Beginner Paused
วางลำตัวและปลายเท้าลงพื้น ลำตัวเป็นเส้นตรงตั้งแต่หัวถึง
ส้นเท้า ลำตัวไม่ทรานเกรงที่ท่าพล
SETS: 3 SECONDS: 60 VOLUME: 3 x 60s
- ID: 6 Deadlift** (Paused)
Legs Advanced Paused
ยืนก้นเก้าอี้เท้าตัก ยกเหล็กจากพื้นด้วยแรงขาและหลัง แขน
ลำตัวงอเล็กน้อยลงเล็กน้อย
SETS: 3 REPS: 5 VOLUME: 3 x 5
- ID: 7 Burpee** (Paused)
Full Body Intermediate Paused
ลงเข่าลงมือเท้าตามลำดับ: ยืน - นั่ง - คลานลงพื้น - ยืน
ขึ้น - ยกตัวขึ้น
SETS: 4 REPS: 10 VOLUME: 4 x 10
- ID: 8 Lunges** (Paused)
Legs Beginner Paused
ก้าวไปข้างหน้า 1 ก้าว สดตัวลงจนขาหน้าลงมือขึ้นพื้น แล้ว
ดันตัวขึ้นขึ้นยืนใหม่
SETS: 3 REPS: 12 VOLUME: 3 x 12
- ID: 9 Push** (Archived)
Arms Beginner Archived
สองข้อมือ
SETS: 10 REPS: 2 VOLUME: 10 x 2

Status Change Notifications (Right Side):

- เปลี่ยนสถานะ "Lunges" เป็น Paused (PATCH)
- เปลี่ยนสถานะ "Burpee" เป็น Paused (PATCH)
- เปลี่ยนสถานะ "Pull-up" เป็น Paused (PATCH)
- เปลี่ยนสถานะ "Plank" เป็น Paused (PATCH)
- เปลี่ยนสถานะ "Deadlift" เป็น Paused (PATCH)

Footer: Workout Routine Catalog — Node.js + Express + Vanilla JS — Memory DB (ข้อมูลยังไม่เก็บใน DB จริง)

ภาพที่ ก-6 Modal ยืนยันการลบ

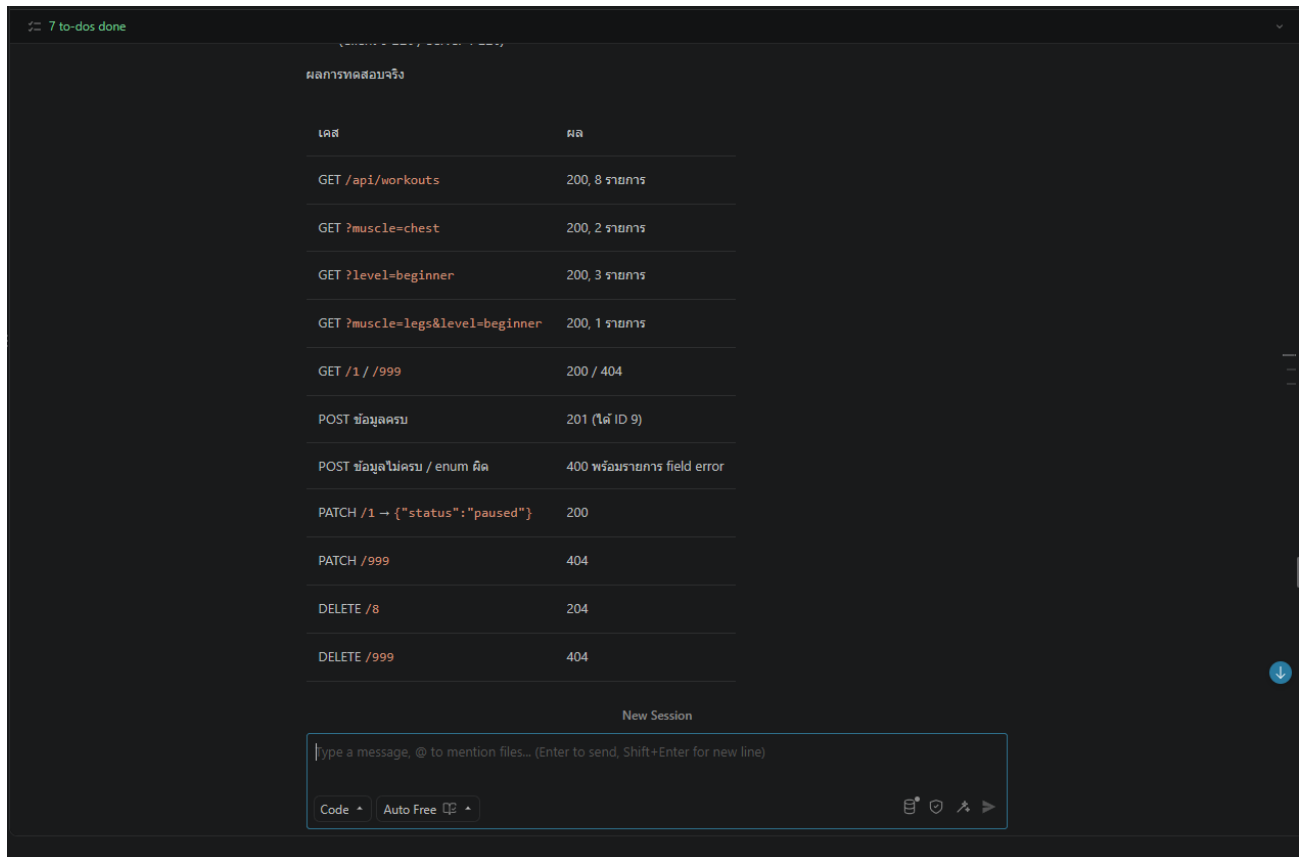


ภาคผนวก ข: ผลการ Debug

README ของโครงการระบุหลักฐานการดีบั๊กไว้ 2 รายการ ได้แก่ Debug 1 - API Health และ Debug 2 - ผลการทดสอบระบบ แต่ไฟล์รูปภาพ debug1.png และ debug2.png ไม่ได้ถูกแนบมาในไฟล์ชิ้นงานที่ส่งมาพร้อมคำขอนี้

จึงเว้นตำแหน่งไว้สำหรับใส่ภาพจริง

Debug 1 - API Health



The screenshot shows a REST client interface with a table of test results. The table has two columns: 'เคส' (Case) and 'ผล' (Result). The tests include various GET, POST, PATCH, and DELETE requests to an API, with their corresponding status codes and messages. At the bottom, there is a 'New Session' button and a text input field for sending messages.

เคส	ผล
GET /api/workouts	200, 8 รายการ
GET ?muscle=chest	200, 2 รายการ
GET ?level=beginner	200, 3 รายการ
GET ?muscle=legs&level=beginner	200, 1 รายการ
GET /1 / /999	200 / 404
POST ข้อมูลครบ	201 (ได้ ID 9)
POST ข้อมูลไม่ครบ / enum ผิด	400 พร้อมรายการ field error
PATCH /1 → {"status": "paused"}	200
PATCH /999	404
DELETE /8	204
DELETE /999	404

New Session

Type a message, @ to mention files... (Enter to send, Shift+Enter for new line)

Code Auto Free

Debug 2 - ผลการทดสอบระบบ

